

Lab 15: Migrate a PostgreSQL Schema and Live Data with MOLT (55 min)

Learning Objectives

By the end of this lab you will be able to:

- Convert a PostgreSQL schema to CockroachDB and fix every incompatibility it surfaces
- **Redesign primary keys during the migration** — the one chance you get to fix distribution for free
- Move bulk data with `MOLT Fetch` and verify it with `MOLT Verify`
- Run a dual-write / shadow-read cutover and know the rollback point at each step
- Compare query plans on both sides and catch the queries that regress
- Decide when *not* to migrate

Prerequisites

- **Docker Desktop** (or Docker Engine) running — there is no `cockroach` binary to install
- `psql` client
- Nothing else to install: PostgreSQL and the MOLT tools both run as containers.

MOLT in a container, like everything else. Setup step 1 defines a `molt` alias that runs the `cockroachdb/molt` image on the cluster network. Because it runs *inside* that network, the `molt` commands use `$PG_NET` and `$CRDB_NET` (container names) rather than the `localhost` URLs your own `psql` uses. A pure-SQL fallback is given below if you would rather skip MOLT entirely.

Setup

Stay in the repository root for the whole lab — every command is written to run from there.

1. Cluster, working directory, MOLT

```
scripts/crdb up
mkdir -p /tmp/lab15
alias molt='docker run --rm --network crdb-labs_default -v /tmp/lab15:/tmp/lab15 cockroachdb/
    molt'
```

(Re-run the `alias` line in any new terminal you open.)

2. Source PostgreSQL

It joins the cluster's network so MOLT — and CockroachDB itself — can reach it by name:

```
docker run -d --name lab15-pg --network crdb-labs_default -p 5432:5432 \
  -e POSTGRES_PASSWORD=pg -e POSTGRES_DB=legacy \
  postgres:16 -c wal_level=logical

sleep 8
export PG='postgresql://postgres:pg@localhost:5432/legacy'      # from your machine
export PG_NET='postgresql://postgres:pg@lab15-pg:5432/legacy'   # from inside a container
```

3. A realistically bad legacy schema

```
psql "$PG" <<'SQL'
CREATE TABLE customers (
  id          SERIAL PRIMARY KEY,          -- sequential PK: hotspot waiting to happen
  email       VARCHAR(255) NOT NULL UNIQUE,
  name        VARCHAR(255) NOT NULL,
  tenant_id   INTEGER NOT NULL,
  created_at  TIMESTAMP DEFAULT now()
);

CREATE TABLE orders (
  id          SERIAL PRIMARY KEY,
  customer_id INTEGER NOT NULL REFERENCES customers(id),
  tenant_id   INTEGER NOT NULL,
  total       NUMERIC(12,2) NOT NULL,
  status      VARCHAR(32) NOT NULL DEFAULT 'new',
  metadata    JSON,
  created_at  TIMESTAMP DEFAULT now()
);

CREATE TABLE order_events (
  id          BIGSERIAL PRIMARY KEY,      -- append-only, monotonic: the worst case
  order_id    INTEGER NOT NULL REFERENCES orders(id),
  event_type  VARCHAR(64) NOT NULL,
  occurred_at TIMESTAMP DEFAULT now()
);

CREATE INDEX idx_orders_status  ON orders(status);      -- low cardinality
CREATE INDEX idx_orders_created ON orders(created_at);
CREATE INDEX idx_events_order   ON order_events(order_id);

-- A stored procedure and a trigger: neither survives the migration as-is
CREATE OR REPLACE FUNCTION touch_order() RETURNS TRIGGER AS $$
BEGIN NEW.created_at = now(); RETURN NEW; END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER orders_touch BEFORE INSERT ON orders
  FOR EACH ROW EXECUTE FUNCTION touch_order();

INSERT INTO customers (email, name, tenant_id)
SELECT 'user' || g || '@example.com', 'User ' || g, (g % 20) + 1
FROM generate_series(1, 20000) g;

INSERT INTO orders (customer_id, tenant_id, total, status, metadata)
SELECT (random()*19999)::int + 1, (g % 20) + 1, (random()*500)::numeric(12,2),
       (ARRAY['new', 'paid', 'shipped', 'cancelled'])[1 + (g % 4)],
       ('{"src":"web","seq":"' || g || '"}'):json
FROM generate_series(1, 100000) g;

INSERT INTO order_events (order_id, event_type)
SELECT (random()*99999)::int + 1, (ARRAY['created', 'paid', 'shipped'])[1 + (g % 3)]
FROM generate_series(1, 200000) g;
SQL

psql "$PG" -c "SELECT count(*) FROM customers; SELECT count(*) FROM orders; SELECT count(*)
FROM order_events;"
```

4. Target CockroachDB

```
scripts/crdb sql -e "CREATE DATABASE target;"
export CRDB='postgresql://root@localhost:26257/target?sslmode=disable'      # from your
machine
export CRDB_NET='postgresql://root@crdb1:26257/target?sslmode=disable'      # from inside a container
export STAGE_NET='postgresql://root@crdb1:26257/stage?sslmode=disable'      # Part C's
staging database
```

The SQL blocks below assume a shell open **in the target database**:

```
scripts/crdb sql -d target
```

The cluster runs in Docker (see [Lab 1](#)). From your machine it is `localhost:26257`; from inside another container it is `crdb1:26257`. `scripts/crdb run ...` executes inside node 1.

Tasks

Part B: Redesign the Schema for Distribution (20 min)

This is the highest-value 20 minutes of the migration. Changing a primary key after cutover means another migration; changing it now costs nothing.

1. Score the legacy schema against the Playbook:

Legacy design	Problem	Playbook fix
<code>customers.id SERIAL</code>	Rightmost-range write hotspot	UUID PK, or <code>(tenant_id, id)</code>
<code>orders.id SERIAL</code>	Same, plus tenant queries scan everywhere	#3 Per-Tenant Co-located PK
<code>order_events.id BIGSERIAL</code>	Pure append at max rate — worst case	#1 Hash-Sharded Time-Series PK
<code>idx_orders_status</code>	Low-cardinality index, rarely selective	#6 Hot/Cold partial index
Nothing ages out <code>order_events</code>	Unbounded growth	#9 TTL table

2. Write the target schema:

```

USE target;

CREATE TABLE customers (
  id          UUID NOT NULL DEFAULT gen_random_uuid(),
  tenant_id   INT NOT NULL,
  email       STRING NOT NULL,
  name        STRING NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  legacy_id   INT NOT NULL,           -- keep the old key for the cutover
  PRIMARY KEY (tenant_id, id),       -- Playbook #3
  UNIQUE INDEX customers_email_key (email),
  UNIQUE INDEX customers_legacy_id_key (legacy_id)
);

CREATE TABLE orders (
  id          UUID NOT NULL DEFAULT gen_random_uuid(),
  tenant_id   INT NOT NULL,
  customer_id UUID NOT NULL,
  total       DECIMAL(12,2) NOT NULL,
  status      STRING NOT NULL DEFAULT 'new',
  metadata    JSONB,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  legacy_id   INT NOT NULL,
  PRIMARY KEY (tenant_id, id),       -- Playbook #3
  UNIQUE INDEX orders_legacy_id_key (legacy_id),
  INDEX orders_customer_idx (tenant_id, customer_id),
  -- Playbook #6: only "open" orders are queried by status
  INDEX orders_open_idx (tenant_id, created_at DESC)
  STORING (total, customer_id) WHERE status IN ('new', 'paid')
);

CREATE TABLE order_events (
  order_id    UUID NOT NULL,
  occurred_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  id          UUID NOT NULL DEFAULT gen_random_uuid(),
  event_type  STRING NOT NULL,
  legacy_id   INT NOT NULL,
  -- Playbook #1: hash-shard the time-ordered append stream
  PRIMARY KEY (order_id, occurred_at, id) USING HASH WITH (bucket_count = 16)
) WITH (ttl_expire_after = '180 days', ttl_job_cron = '@daily'); -- Playbook #9

```

Note what is missing: no foreign keys yet. FK validation during a bulk load is expensive and serializing. Load first, add constraints after — see Part C step 5.

Note legacy_id: during dual-write and shadow-read you must be able to match a row on both sides. Keep the old key until cutover is complete, then drop the column.

3. **Predict the improvement before you measure it.** Which of these is now cheaper, and why?

```
-- Q1: all of one tenant's orders
SELECT * FROM orders WHERE tenant_id = 7 ORDER BY created_at DESC LIMIT 50;

-- Q2: one order's event history
SELECT * FROM order_events WHERE order_id = '...' ORDER BY occurred_at DESC;

-- Q3: open orders across all tenants
SELECT count(*) FROM orders WHERE status = 'new';
```

Q1 and Q2 get faster (co-located, index-covered). Q3 gets *slower* — it now fans out across every tenant's ranges. If Q3 is a hot query, `(status, tenant_id)` deserves its own index — **but check whether the query is actually needed** before adding write amplification for it.

Part C: Move the Data with MOLT Fetch (20 min)

The primary keys changed, so this is a *transformation*, not a copy — which is why Part B kept `legacy_id`. The move is therefore two steps: land the legacy tables **as they are** in a staging database, then transform them into the new key space with SQL.

1. **Land the legacy tables with MOLT Fetch.** It creates the staging tables itself (`--table-handling drop-on-target-and-recreate`), with the source's column types:

```
scripts/crdb sql -e "CREATE DATABASE stage;"

molt fetch \
  --source "$PG_NET" \
  --target "$STAGE_NET" \
  --table-filter 'customers|orders|order_events' \
  --table-handling drop-on-target-and-recreate \
  --allow-tls-mode-disable \
  --direct-copy
```

The warnings about "stats based sharding" are expected on a small table.

Flag	Purpose
<code>--direct-copy</code>	Stream via <code>COPY</code> , no intermediate object store
<code>--table-handling drop-on-target-and-recreate</code>	Let MOLT create the target tables from the source schema
<code>--bucket-path s3://...</code>	Stage as CSV in object storage for <code>IMPORT INTO</code> (faster for large data)
<code>--allow-tls-mode-disable</code>	Required because the lab cluster is <code>--insecure</code>
<code>--table-filter</code>	Regex of tables to move

```
scripts/crdb sql -d stage -e
  "SELECT count(*) FROM customers; SELECT count(*) FROM orders; SELECT count(*) FROM
  order_events;"
```

2. **Without MOLT (the portable path)** — same result via CSV, if you would rather see every step. Skip this if step 1 worked.

```
psql "$PG" -c "\copy (SELECT id, tenant_id, email, name, created_at FROM customers) TO '/tmp/lab15/customers.csv' CSV"
psql "$PG" -c "\copy (SELECT id, customer_id, tenant_id, total, status, metadata::text, created_at FROM orders) TO '/tmp/lab15/orders.csv' CSV"
psql "$PG" -c "\copy (SELECT id, order_id, event_type, occurred_at FROM order_events) TO '/tmp/lab15/order_events.csv' CSV"
```

```
scripts/crdb sql -d stage <<'SQL'
CREATE TABLE customers (id INT PRIMARY KEY, tenant_id INT, email STRING, name STRING, created_at TIMESTAMPTZ);
CREATE TABLE orders (id INT PRIMARY KEY, customer_id INT, tenant_id INT, total DECIMAL(12,2), status STRING, metadata JSONB, created_at TIMESTAMPTZ);
CREATE TABLE order_events (id INT PRIMARY KEY, order_id INT, event_type STRING, occurred_at TIMESTAMPTZ);
SQL

# The CSVs are on your machine; the node is in a container and cannot see them. Copy each one in, then stage it in the cluster's userfile store.
for t in customers orders order_events; do
  scripts/crdb cp /tmp/lab15/$t.csv crdb1:/tmp/$t.csv
  scripts/crdb run userfile upload /tmp/$t.csv /lab15/$t.csv --insecure
done

scripts/crdb sql -d stage <<'SQL'
IMPORT INTO customers CSV DATA ('userfile:///lab15/customers.csv');
IMPORT INTO orders CSV DATA ('userfile:///lab15/orders.csv');
IMPORT INTO order_events CSV DATA ('userfile:///lab15/order_events.csv');
SQL
```

3. **Transform staging into the new key space** (in your `target` shell):

```
INSERT INTO customers (tenant_id, email, name, created_at, legacy_id)
SELECT tenant_id, email, name, created_at, id FROM stage.customers;

INSERT INTO orders (tenant_id, customer_id, total, status, metadata, created_at, legacy_id)
SELECT c.tenant_id, c.id, s.total, s.status, s.metadata, s.created_at, s.id
FROM stage.orders s JOIN customers c ON c.legacy_id = s.customer_id;

INSERT INTO order_events (order_id, occurred_at, event_type, legacy_id)
SELECT o.id, s.occurred_at, s.event_type, s.id
FROM stage.order_events s JOIN orders o ON o.legacy_id = s.order_id;
```

4. **Verify — twice.** MOLT Verify compares tables of the same shape, so point it at the staging copy; the transformed `target` tables are checked with counts and a business checksum:

```
molt verify --source "$PG_NET" --target "$STAGE_NET" --allow-tls-mode-disable --table-filter 'customers|orders|order_events'
```

```

for t in customers orders order_events; do
  echo -n "$t pg="; psql "$PG" -tAc "SELECT count(*) FROM $t"
  echo -n "      crdb="; scripts/crdb sql -d target --format=tsv -e "SELECT count(*) FROM
    $t" | tail -1
done

# Business checksum, not just a row count
psql "$PG" -tAc "SELECT sum(total)::numeric(20,2) FROM orders"
scripts/crdb sql -d target --format=tsv -e "SELECT sum(total)::DECIMAL(20,2) FROM orders"
  | tail -1

```

5. Add constraints after the load, not before:

```

ALTER TABLE orders ADD CONSTRAINT orders_customer_fk
  FOREIGN KEY (tenant_id, customer_id) REFERENCES customers (tenant_id, id);

SELECT job_id, description, status FROM [SHOW JOBS]
WHERE job_type IN ('SCHEMA CHANGE', 'NEW SCHEMA CHANGE') ORDER BY created DESC LIMIT 3;

DROP DATABASE stage CASCADE;  -- the staging copy has done its job

```

Time this step. FK validation is a full scan of both tables — on a real dataset it is a scheduled maintenance item, not an afterthought.

Part D: Compare Plans — Catch the Regressions (15 min)

Migrations regress queries. Find out which ones before your users do.

1. Same query, both engines:

```

psql "$PG" -c "EXPLAIN ANALYZE SELECT * FROM orders WHERE tenant_id = 7 ORDER BY
  created_at DESC LIMIT 50;"
scripts/crdb sql -d target -e "EXPLAIN ANALYZE SELECT * FROM orders WHERE tenant_id = 7
  ORDER BY created_at DESC LIMIT 50;"

```

2. Run the comparison across a query set:

Query	PG time	CRDB time	CRDB rows read	Verdict
Tenant order list				
Order event history				
Count by status (all tenants)				
Customer by email				
Join orders → customers				

3. Statistics matter. A freshly loaded table has no useful statistics until they're collected:

```

SHOW STATISTICS FOR TABLE orders;
CREATE STATISTICS orders_stats ON tenant_id, status, created_at FROM orders;
--          ^^^^^^^^^^^ the statistics name is required

```

Re-run the plans. Several will change. **Always collect statistics before benchmarking a migration** — otherwise you are measuring the optimizer flying blind.

4. The regressions to expect, and their fixes:

Symptom	Cause	Fix
Full scan where PG used an index	Missing/stale statistics, or index not carried over	<code>CREATE STATISTICS</code> , recreate index
Slow multi-row <code>IN</code> lookup	Keys scattered across ranges	Reorder PK to co-locate
Slow <code>COUNT(*)</code> on a big table	Distributed scan, no shortcut	<code>AS OF SYSTEM TIME</code> follower read, or maintain a counter
Slow FK-heavy inserts	Each FK is a distributed read	Batch inserts; consider dropping non-critical FKs
High p99 with fine p50	Contention or cross-range transactions	Lab 5/Lab 8 techniques

Optional — If Time Allows

These parts are not required to complete the lab; they extend it by about 35 minutes. Do them if you finish early, or after class — the cluster and data from the core parts carry over.

Part A: Dialect Differences — What Actually Breaks (15 min)

1. Dump the source schema and try it verbatim:

```
pg_dump "$PG" --schema-only --no-owner --no-privileges > /tmp/lab15/schema.sql
scripts/crdb sql -e "CREATE DATABASE dialect;" # scratch database, so nothing
collides with target
scripts/crdb sql -d dialect < /tmp/lab15/schema.sql 2>&1 | tee /tmp/lab15/errors.log
grep -i error /tmp/lab15/errors.log | head -20
```

2. The cheat sheet. Every row here is something you will hit:

PostgreSQL	CockroachDB	Action
<code>SERIAL</code> / <code>BIGSERIAL</code>	Works, but means <code>unique_rowid()</code>	Change to <code>UUID DEFAULT gen_random_uuid()</code>
Sequences (<code>nextval</code>)	Supported but serialize on one range	Avoid on hot paths; use UUID
<code>plpgsql</code> triggers	Not supported	Move to application code or a UDF where possible
Stored procedures	UDFs only (<code>CREATE FUNCTION</code>)	Rewrite
<code>JSON</code>	Stored as <code>JSONB</code>	Fine — inverted-index it if you query into it
<code>VARCHAR(n)</code>	Supported; <code>STRING</code> is idiomatic	Cosmetic
<code>TIMESTAMP</code>	Prefer <code>TIMESTAMPTZ</code>	Change — timezone bugs are forever

PostgreSQL	CockroachDB	Action
text search (tsvector)	Full-text search differs	Re-test
PostGIS extension	Spatial is built in, syntax mostly compatible	Re-test
SELECT ... FOR UPDATE	Supported	Behaviour differs under SERIALIZABLE
Foreign keys	Supported; each FK check is a distributed read	Audit hot-path FKs
ON DELETE CASCADE	Supported	Cascades can be surprisingly expensive
Materialized views	Supported, manual refresh	Re-test refresh cost
Table inheritance	Not supported	Redesign
pg_* extensions	Not supported	Find built-in equivalent

3. **Ask the source what it uses** — do this before you promise a timeline:

```
psql "$PG" -c "SELECT count(*) FROM information_schema.triggers;"
psql "$PG" -c "SELECT proname, prolang::regtype FROM pg_proc WHERE pronamespace =
'public'::regnamespace;"
psql "$PG" -c "SELECT extname FROM pg_extension;"
psql "$PG" -c "SELECT sequencename FROM pg_sequences;"
```

Part E: Zero-Downtime Cutover (15 min)

The pattern, with the rollback point named at each step:

Phase 1	Schema in place on CRDB, PG live	← rollback: drop CRDB
Phase 2	Bulk load + continuous replication (MOLT Fetch)	← rollback: stop replication
Phase 3	Dual write: app writes both, PG authoritative	← rollback: stop CRDB writes
Phase 4	Shadow reads: read both, compare, serve PG	← rollback: stop shadow reads
Phase 5	Flip: CRDB authoritative, PG still written	← rollback: flip back (minutes)
Phase 6	Stop writing PG, decommission	← rollback: restore from backup

1. **Continuous replication.** MOLT Fetch does the bulk load (Part C); keeping the two in sync afterwards is the separate MOLT *Replicator* tool, which tails PostgreSQL's logical replication slot (that is why the source was started with `wal_level=logical`). It is not part of this lab — the point of the phase is that a rollback is "stop replication", nothing more. Read the phase table above and move on to step 2.

2. **Dual write, in application code:**

```
def create_order(pg_conn, crdb_conn, order):
    with pg_conn: # authoritative during phase 3
        pg_id = insert_pg(pg_conn, order)
    try:
        insert_crdb(crdb_conn, order, legacy_id=pg_id)
    except Exception as e:
        # NEVER fail the user request on the shadow write.
        metrics.increment("dual_write.crdb_failure")
        log.warning("crdb dual write failed: %s", e)
    return pg_id
```

3. Shadow reads — compare, serve the authoritative one, alert on divergence:

```
def get_order(pg_conn, crdb_conn, order_id):
    pg_row = read_pg(pg_conn, order_id)
    try:
        crdb_row = read_crdb(crdb_conn, order_id)
        if normalize(crdb_row) != normalize(pg_row):
            metrics.increment("shadow_read.divergence")
            log.warning("divergence on %s", order_id)
    except Exception:
        metrics.increment("shadow_read.error")
    return pg_row # PG still serves the user
```

4. The gate for flipping. Do not flip on a schedule; flip on evidence:

☐

Replication lag < 1 s sustained for 24 h

☐

Shadow-read divergence rate = 0 over 24 h

☐

Every query in the comparison table at or under its PG latency

☐

A tested rollback path with a measured RTO (Lab 11)

☐

Backups running on the target, with a restore drill completed

5. After cutover:

```
SET sql_safe_updates = false; -- the shell refuses DROP COLUMN otherwise
ALTER TABLE customers DROP COLUMN legacy_id;
ALTER TABLE orders DROP COLUMN legacy_id;
ALTER TABLE order_events DROP COLUMN legacy_id;
```

Part F: When NOT to Migrate (5 min)

Argue both sides for each. The honest answer is sometimes "stay on PostgreSQL".

Workload	Migrate?	Why
Single-region app, 50 GB, one PG instance is fine		
Heavy analytical scans and window functions over the full dataset		
Deep dependence on PostGIS-specific functions or extensions		
Needs multi-region low-latency reads with strong consistency		
Cannot tolerate a maintenance window for failover		
Team of two with no distributed-systems experience		
Write throughput exceeds one machine		

The honest guidance: CockroachDB earns its complexity when you need **survivability**, **horizontal write scale**, or **data locality**. If you need none of those, PostgreSQL is simpler and you should keep it.

Cleanup

```
docker rm -f lab15-pg
scripts/crdb down
```

Lab 15 Deliverables

□ **Dialect audit** of the source, with every incompatibility identified before migration □ **Redesigned schema** applying at least four Playbook patterns, with `legacy_id` for cutover □ **Data moved** and verified by row count *and* business checksum □ **Constraints added post-load**, with the validation cost measured □ **Plan comparison** across a query set, with regressions found and fixed □ **Cutover plan** with a named rollback point at every phase and an evidence-based flip gate □ **A defended "don't migrate" case**

Challenge Exercises

1. **Trigger replacement.** The source has a `BEFORE INSERT` trigger. Implement the same behaviour three ways — column `DEFAULT`, a CockroachDB UDF, and application code — and argue for one.
2. **Migrate the sequence semantics.** Some downstream system depends on monotonically increasing order IDs. You just replaced them with UUIDs. Solve it without reintroducing the hotspot (hint: a per-tenant sequence, or a sortable UUID like ULID).
3. **Verify continuously.** Write a script that samples 1,000 random rows from both sides every minute during dual-write and reports divergence. What sampling rate gives you confidence at your data volume?
4. **Measure the FK tax.** Load `order_events` with and without the FK to `orders`. How much slower is the constrained load, and is the constraint worth it here?

Reference

Command	Purpose
<code>molt fetch --direct-copy --allow-tls-mode-disable</code>	Bulk load source → target
<code>molt verify</code>	Row-by-row source/target comparison
<code>pg_dump --schema-only</code>	Extract the source schema
<code>IMPORT INTO ... CSV DATA (...)</code>	Fast bulk load into CockroachDB
<code>cockroach userfile upload</code>	Stage a file for <code>IMPORT</code>
<code>CREATE STATISTICS ON ... FROM t</code>	Collect stats before benchmarking
<code>ALTER TABLE ... ADD CONSTRAINT ... FOREIGN KEY</code>	Add FKs after the load
<code>SHOW JOBS</code>	Watch validation and schema-change jobs